

1. 源程序到目标程序的过程叫什么？分别有哪几种类型？

答：源程序到目标程序的过程叫 翻译

翻译程序分为：解释器（解释程序）、编译器（编译程序）、汇编器（汇编程序）

2. 冯诺依曼计算机有哪些部分组成，各自功能是什么？

答：运算器、控制器、存储器、输入设备和输出设备

- 运算器：是对数据进行运算的部件,其主要功能是对二进制数据进行算术运算(加、减、乘、除)和逻辑运算(与、或、非、移位),故又称为算术逻辑单元;
- 控制器：是整个计算机的控制中心,其功能是控制计算机各个部件自动协调工作。
- 存储器：用来存放程序和数据;
- 输入设备：用来向计算机输入程序和原始数据;
- 输出设备：用来输出计算机的处理结果及程序;

3. 指令由那几部分组成，什么决定了指令的操作类型？

答：指令通常被划分成若干字段，有操作码和地址码

操作码：指出指令的操作类型，如取数、存数、加、减、传送、跳转等

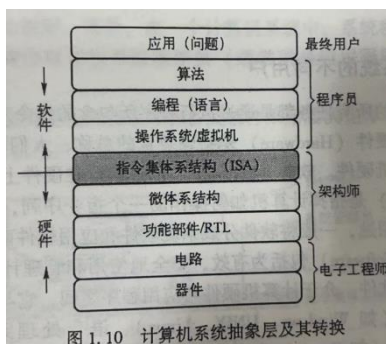
地址码：指出指令所处的操作数的地址，如寄存器编号、主存单元编号等

4. 什么是“存储程序”工作方式？

答：(1) 事先编制程序 (2) 实现存储程序 (3) 自动、连续地执行程序

即事先编写程序,再由计算机把这些信息存储起来,然后连续地、快速地执行程序,从而完成各种运算过程。

5. 计算机系统的层次结果如何划分？计算机系统的用户可分为哪几类？每类用户工作在哪个层次？



- 计算机系统为软件和硬件两部分，软件和硬件的界面是指令及体系结构(ISA)。
- 计算机系统从高到底粗分为应用软件、系统软件和硬件三个层次：
- 不同计算机用户工作在不同的层次，从高到低细分为应用程序级（最终用户）、高级语言虚拟机级（高级语言程序员或应用程序员）、汇编语言虚拟机级（汇编语言程序员）、操作系统虚拟机级（系统管理员）、机器语言机器级（机器语言程序员）

6. 程序的 CPI 与哪些因素有关？

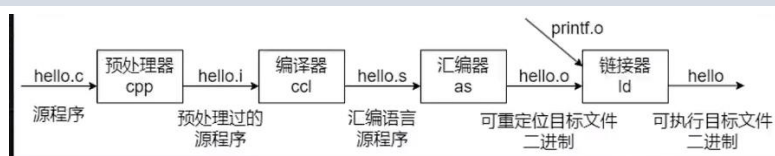
$CPI = \text{程序总时钟周期数} / \text{程序总指令条数}$ ，CPI 与时钟周期、指令条数有关。在指令条数和时钟周

期一定的情况下，CPI 越大，执行时间越长

7. 为什么说性能指标 MIPS 不能很好的反映计算机的性能？

答：MIPS 反映了机器执行定点指令的速度，但是用 MIPS 来对不同的机器进行性能比较有时是不准确或不客观的。因为不同的机器的指令集不同，指令的功能也不同。也许一个机器上一条指令的功能，在另外一个机器上要用多条指令来完成，这样，同样的指令条数所完成的功能可能完全不同。

8. 源程序到可执行目标文件转换过程



阶段	处理程序	处理结果（扩展名）
预处理阶段	预处理器	源程序（.i）
编译阶段	编译器	汇编源程序（.s）
汇编阶段	汇编器	可重定位目标文件二进制（.o）
链接阶段	链接器	可执行文件二进制

四步：

预处理阶段-->编译阶段-->汇编阶段-->链接阶段

9 为什么计算机内部采用二进制表示信息？

- ① 技术实现简单
- ② 抗干扰能力强，可靠性高
- ③ 运算规则简单
- ④ 适合逻辑运算
- ⑤ 易于进行转换

10 既然计算机内部所有信息都用二进制表示，为什么还要用到十六进制或八进制数？

答：采用八进制和十六进制数主要是编程序时缩短对二进制数的写法

11 在浮点数的基数和总位数一定的情况下，浮点数的表示范围和精度分别由什么决定？

答：阶码位数、尾数位数

12 为什么要对浮点数进行规格化？有哪两种规格化操作？

答：由于规格化浮点数具有惟一的表示形式，而且在计算机中尾数能获得最大的有效数字，所以在一般的计算机中选用规格化浮点运算。规格化操作包括：左规处理和右规处理

13 指令集体系结构 ISA 规定的内容

- (1) 指令格式、操作类型
- (2) 操作数的类型
- (3) 寄存器的名称、编号、长度、用途等
- (4) 操作数能存放的存储空间大小和编址方式
- (5) 大端序还是小端序
- (6) 寻址方式
- (7) 指令执行过程的控制方式，包括程序计数器

14 高级语言转换成机器语言需要哪些步骤

- (8) 预处理成 hello.i (源码)
- (9) 编译成汇编语言 hello.s (源码)
- (10) 汇编成可重定位目标文件 (二进制)
- (11) 链接成可执行目标文件 (二进制)

15 gcc 生成器代码

- (12) 一步到位: `gcc -o1 hello.c hello.c -o hello`
- (13) 预处理: `gcc -E hello1.c -o hello1.i`
- (14) 编译: `gcc -S hello1.i -o hello1.o`
- (15) 汇编: `gcc -c hello1.s -o hello1.o`
- (16) 链接: `gcc hello1.o hello2.o -o hello`

16C 语言程序级别表示有哪几种?

答: 过程调用、选择语句、循环语句

17 简述过程调用 P 调用 Q 的过程

- (17) P 将实参放到 Q 能取到的地方
- (18) P 将返回地址压栈，然后把控制转移到 Q

(19) Q 保护 P 的现场，给自己的内部非静态局部变量分配空间

(20) 执行 Q 的函数体

(21) Q 恢复 P 的现场，释放局部变量所占空间

(22) Q 取出返回地址，把控制转移到 P

18 一条操作指令通常由哪些字段组成?各字段含义是什么?

答：通常情况下，一条指令要由操作码和操作数两部分内容组成。

其中第一部分是指令的操作码，它确定了本条指令时执行算术、逻辑、读写等多种操作中的哪一种功能，计算机为每条指令分配了一个确定的操作码

第二部分是指令的操作数地址，用于给出被操作的信息（指令或数据）的地址，包括参加运算的一或多个操作数所在的地址，运算结果的保存地址，程序的转移地址、被调用的子程序的入口地址等

19 IA-32 中的逻辑运算指令如何生成条件标志? 移位指令可能会改变哪些特殊条件标志?

IA-32 的逻辑运算指令，如 AND、OR、XOR、NOT 等，会根据运算的结果设置条件标志。这些标志包括：

OF（溢出标志）：逻辑运算不会改变 OF 标志，因为逻辑运算不会导致溢出。

SF（符号标志）：根据运算结果的最高位（符号位）设置。如果结果最高位为 1，则 SF 置 1；否则 SF 置 0。

ZF（零标志）：如果运算结果为 0，则 ZF 置 1；否则 ZF 置 0。

AF（辅助进位标志）：逻辑运算不会改变 AF 标志。

PF（奇偶标志）：根据运算结果中 1 的个数设置。如果 1 的个数是偶数，则 PF 置 1；否则 PF 置 0。

CF（进位标志）：逻辑运算不会改变 CF 标志，因为逻辑运算不涉及进位。

移位指令与特殊条件标志

IA-32 的移位指令，除了设置条件标志外，还可能影响以下特殊条件标志：

OF（溢出标志）：算术移位（SAR 和 SAL）可能会影响 OF 标志。当执行算术右移时，如果最高位（符号位）为 1 且移位后不为 0，则 OF 置 1；否则 OF 置 0。对于算术左移，OF 的行为取决于处理器的实现，但通常不会改变 OF 标志。

SF、ZF、PF、CF：这些标志的行为与逻辑运算指令类似，根据移位运算的结果设置。

20 执行条件跳转指令时所用到的条件标志信息从何而来？请举例说明

```
int a = 5;
int b = 10;
if (a == b) {
    // do something
} else {
    // do something else
}
```

当这段代码被编译成IA-32汇编代码时，它可能会包含类似以下的指令：

```
mov eax, 5    ; 将整数5加载到eax寄存器中
mov ebx, 10   ; 将整数10加载到ebx寄存器中
cmp eax, ebx  ; 比较eax和ebx寄存器的值
je equal      ; 如果相等（ZF标志被设置），则跳转到标签'equal'处
; do something else 的代码
jmp endif     ; 无条件跳转到endif标签处
equal:
; do something 的代码
endif:
```

在上面的汇编代码中，`cmp eax, ebx`指令会比较`eax`和`ebx`寄存器的值。如果它们相等，那么ZF（零标志）将被设置。随后，`je equal`指令会检查ZF标志。如果ZF被设置（即`eax`等于`ebx`），则处理器会跳转到标签`equal`处执行代码。如果ZF未被设置，处理器将跳过`equal`标签处的代码并执行`do something else`的代码，然后无条件跳转到`endif`标签处。

注意，`cmp`指令实际上执行了减法操作（`ebx - eax`），但并不存储结果，只是根据结果设置了条件标志。这是为了设置条件标志而进行比较操作的一种常见方法。

总之，条件跳转指令所使用的条件标志信息是由最近的算术或逻辑指令操作后所设置的状态标志提供的。这些标志用于控制程序的流程，使得程序能够根据条件执行不同的代码路径。

21 无条件跳转指令和调用指令的相同点和不同点是什么？

【相同点】：

两者都可以改变程序的执行流程，使程序跳转到指定的地址执行 1。

【不同点】：

无条件跳转指令是直接使程序的控制流转移到指定的目标，而调用指令是使控制流从一个程序段跳转到另一个程序段执行，执行完毕后，必须返回到调用它的程序段。

无条件跳转指令不需要保存返回地址，而调用指令需要保存返回地址，以供被调用程序执行完毕后返回调用程序继续执行 1。

此外，无条件跳转指令和调用指令还有其他不同点，如指令格式、指令长度等。

22 按值传递参数和按地址传递参数两种方式有哪些不同点？

- **数据复制：**按值传递时，会创建参数值的一个副本，并将这个副本传递给函数。这意味着函数接收的是参数值的一个副本，而不是原始值。因此，在函数内部对参数值的修改不会影响到原始值。而按地址传递时，

函数接收的是参数的内存地址，而不是参数值的副本。因此，函数内部对参数值的修改会直接影响到原始值。

- **效率**：由于按值传递需要创建参数只值的副本，因此在大型数据结构或对象上，这种方式可能会导致较大的性能开销。而按地址传递则避免了这种开销，因为它传递地址，而不是实际的数据。
- **安全性**：按值传递更安全，因为函数接收的是参数的副本，所以对副本的修改不会影响到原始数据。而按地址传递可能会带来安全风险，因为函数可以直接修改原始数据。
- **函数设计**：在设计函数时，如果希望函数能修改原始数据，那么应该使用按地址传递。如果希望函数不能修改原始数据，那么应该使用按值传递。

23 为什么在递归深度较深时递归调用时间开销和空间开销都会比较大？

【时间开销较大】：

重复计算：在递归过程中，对于相同的子问题可能会被多次计算。例如，在斐波那契数列的递归实现中， $F(n)$ 会被多次计算，尽管这些计算是重复的。这种重复计算导致了时间的浪费。

函数调用开销：每次递归调用函数时，系统需要执行一系列的操作，如保存现场（将当前的程序计数器、栈指针等寄存器的值保存到栈中），将新的参数和返回地址压入栈中，跳转到函数体执行等。这些操作都会消耗一定的时间。

【空间开销较大】：

递归栈：每次递归调用都会在内存中创建一个新的栈帧来保存局部变量、参数和返回地址等信息。当递归深度较大时，会创建大量的栈帧，导致栈空间的大量占用。如果栈空间不足，可能会导致栈溢出错误。

数据复制：在按值传递参数时，每次递归调用都会创建参数值的一个副本。当递归深度较深时，这些副本会占用大量的内存空间。

24 为什么数据在存储器中最好按地址对齐方式存放？

① **提高访问速度**。CPU 访问存储器时，一次访问一个存储单元，若数据按地址对齐方式存放，则 CPU 可以一次读取整个数据，否则需要多次访问才能读取整个数据，降低了访问速度。

② **充分利用存储空间**。若数据不按地址对齐方式存放，则可能会浪费存储空间，因为 CPU 只能访问对齐的

存储单元，无法访问未对齐的存储单元。

③ **简化硬件设计**。若数据按地址对齐方式存放，则可以简化计算机硬件的设计，因为硬件可以针对对齐的数据进行优化，提高运行效率。

④

⑤ 25 计算机内部为何要采用层次结构存储体系？层次结构存储体系如何构成？

· 原因。计算机系统对存储器的要求主要体现在三个方面，分别是存取速度快、存储容量大、位价格低。但很难找到一种存储器能同时满足这三个要求，因此计算机中采用多种速度、容量、位价格各不相同的存储器，将它们合理地组织起来，形成具有层次结构特征的存储系统。这样就能充分发挥各种存储器的优势，形成优势互补，从而使计算机得到一个综合指标满足要求的存储器系统 1。

· 构成。层次化存储体系结构是由多个存储设备构成的，这些设备之间按照一定的逻辑关系排列组合，形成一个层次结构，主要包括缓存层、磁盘层、磁带库层和云存储 126

26 为什么采用地址对齐方式能减少访问 DRAM 中数据的时间

地址对齐(或称为内存对齐)是一种优化内存访问的策略,它可以提高 CPU 访问内存的效率,并减少访问 DRAM (动态随机存取存储器) 中数据的时间。主要有以下几个原因:

- ① 减少 CPU 访问时间
- ② 减少内存访问冲突
- ③ 优化内存带宽利用率
- ④ 简化硬件设计
- ⑤ 兼容性和可移植性、

27 为什么在 CPU 和主存之间引入 cache 能提高 CPU 的访存效率

① **存储器的速度差异**: CPU 的速度远远快于主存 (DRAM)。随着技术的进步, 这种速度差异变得越来越明显。为了弥补这种速度不匹配, 引入了 Cache 作为 CPU 和主存之间的桥梁。Cache 通常使用更快速的存储技术 (如 SRAM), 其访问速度接近 CPU 的速度。

② **局部性原理**: 程序在执行时, 通常会反复访问相同的数据。这些数据在时间和空间上表现出局部性, 即最

近被访问过的数据在未来很可能再次被访问，或者一个地址被访问后，紧接着很可能访问其相邻地址。Cache 利用这种局部性原理，将最近访问过的数据保存在快速存储器中，从而提高访问效率。

③ **减少访存次数**：当 CPU 需要访问数据时，首先检查 Cache 中是否有所需数据。如果 Cache 命中（即所需数据在 Cache 中），则直接从 Cache 中读取，避免了访问主存的延迟。如果 Cache 未命中，则从主存中读取数据，并将其存储在 Cache 中，以备后续访问。因此，通过减少访问主存的次数，Cache 可以显著提高 CPU 的访存效率。

④ **Cache 的层次结构**：为了提高 Cache 的命中率和效率，现代计算机系统通常采用多级 Cache 结构。这种结构包括多个不同大小、速度和访问成本的 Cache 层次，如 L1 Cache（通常分为指令 Cache 和数据 Cache）、L2 Cache 和 L3 Cache 等。每一级 Cache 都位于 CPU 和下一级存储器之间，起到了缓冲和优化的作用。

28 为什么直接映射方式不需要考虑替换策略

直接映射方式不需要考虑替换策略，是因为每一块的位置已经固定，当需要替换时，直接替换掉 Cache 中对应位置的块即可。

直接映射方式的优点是实现简单，缺点是不够灵活，可能在一个“轮回”中有空闲的情况，却不能往里面放数据，且冲突概率高

29 为什么要考虑 cache 的写策略问题

考虑 Cache 的写策略问题，主要是为了保证数据的一致性和提高 Cache 的命中率。

Cache 的写策略主要包括写回法和写直达法。写回法是指当 Cache 中的数据被修改后，不立即将其写回主存，而是等待该数据被替换出去时才写回主存。这种方法可以减少对主存的写操作次数，但可能会增加数据的不一致性。写直达法是指当 Cache 中的数据被修改后，立即将其写回主存。这种方法可以保证数据的一致性，但会增加对主存的写操作次数，降低 Cache 的命中率 1。

因此，在选择 Cache 的写策略时，需要根据具体的应用场景和需求进行权衡和选择。

30 什么是物理地址？什么是逻辑地址？地址转换由硬件还是软件实现？为什么

物理地址：是内存中各存储单元的编号，即存储单元的真实地址，它是可识别、可寻址并实际存在的；

逻辑地址：是指在计算机体系结构中是指应用程序角度看到的内存单元、存储单元、网络主机的地址

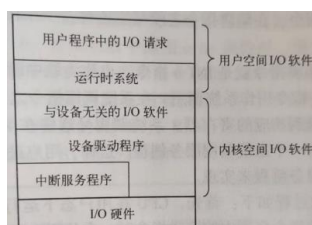
地址转换可以由硬件或软件实现，这取决于具体的计算机系统和架构。在某些情况下，地址转换可能完全由硬件完成，例如使用专门的地址转换单元或内存管理单元（MMU）。在其他情况下，地址转换可能由操作系统和相关的软件支持完成，例如使用页表和段表等数据结构来管理地址空间。

至于为什么地址转换由硬件或软件实现，这取决于多种因素，包括性能、灵活性、可扩展性和成本等。硬件实现通常可以提供更快的地址转换速度，因为硬件可以并行处理多个地址转换请求，并且可以直接与内存和 I/O 设备交互。然而，硬件实现可能更加复杂和昂贵，并且可能不如软件实现灵活和可扩展。

31 在存储器层次化结构中，“cache-主存”“主存-外存”这两个层次有哪些不同？

- ① **解决的问题不同**：Cache-主存层次主要用于解决主存与 CPU 速度不匹配的问题，而主存-外存层次则主要用于解决存储数据的容量问题。
- ② **数据交互方式不同**：Cache-主存层次的数据交互由硬件自动完成，对所有程序员透明；而主存-外存层次的数据交互则由硬件和操作系统协调完成，对应用程序员透明，但对非系统程序员不透明。
- ③ **速度和容量不同**：Cache-主存层次中，Cache 的速度比主存快，容量比主存小；而主存-外存层次中，主存的速度比外存快，容量比外存小。

32 I/O 子系统的层次结构是怎样的？



33 系统调用封装函数对应的机器级代码结构是怎样的

① 用户空间代码：

用户程序包含了对系统调用的封装函数调用，例如 `read()`, `write()`, `fork()` 等。

封装函数通常是库函数，它们将系统调用的参数打包，并准备进行内核调用。

② 陷入内核：

当用户程序执行系统调用时，CPU 需要切换到内核模式（通常通过软件中断实现）。

这通常涉及到更改 CPU 的状态，比如禁用中断，保存用户空间上下文（如程序计数器、栈指针、寄存器

等), 并将这些上下文信息保存到内核栈中。

③ 系统调用表:

操作系统内核维护一个系统调用表, 其中列出了所有可用的系统调用及其对应的处理函数。

CPU 将系统调用的编号 (通常是一个整数) 与系统调用表进行匹配, 找到相应的处理函数。

④ 系统调用处理函数:

一旦找到匹配的系统调用处理函数, CPU 就会执行这个函数。

处理函数通常包含检查参数有效性、执行实际的内核操作 (如文件操作、内存管理等) 以及返回结果给调用者等步骤。

⑤ 返回用户空间:

系统调用完成后, 内核需要恢复用户空间的上下文, 这包括恢复之前保存的寄存器值、栈指针等。

CPU 重新启用中断, 并将控制权返回给用户程序。

⑥ 用户空间继续执行:

用户程序继续执行, 处理系统调用的结果。

34 为什么系统调用的开销很大?

① **用户态与内核态的切换**: 程序要读取磁盘上的数据只能通过系统调用 `read`, 而 `read` 的代码在内核中, 所以当 `read` 调用发生时, 执行权会从用户代码转移到内核代码, 执行内核代码需要消耗时间。

② **CPU 要做的事情太多**: 系统调用指令本身的开销比一般的 `CALL` 和 `JMP` 要多一些, 因为同时又要进行一些额外的检查 (权限、有效性等)。

③ **软件要做的事情太多**: 除了 `syscall` 本身的开销外, 对于抢占的系统, 中断、高优先级的任务调度、线程间的同步、IO 等待等等都可能会造成各种开销。

35C 标准 I/O 库函数是在用户态执行还是在内核态执行

用户态执行

36 与 I/O 操作相关的系统调用封装函数是在用户态执行还是在内核态执行

用户态执行

37 什么是程序直接控制 I/O 放松？说明其工作原理

程序直接控制 I/O 方式是一种最基本的 I/O 控制方式，也称为轮询或查询方式 12。

工作原理是：计算机从外部设备读取数据到存储器，每次读一个字的数据。对读入的每个字，CPU 需要对外设状态进行循环检查（轮询），直到确定该字已经在 I/O 控制器的数据寄存器中。

38 为什么在保护现场和恢复现场的过程中，CPU 必须关中断

- ① 防止数据不一致
- ② 保护上下文信息
- ③ 防止死锁和递归中断
- ④ 确保操作的原子性

39 什么是中断控制 I/O 方式？说明其工作原理

中断控制 I/O 方式是一种基于中断机制的 I/O 控制方式。当中断控制 I/O 方式工作时，CPU 会发出启动命令，启动 I/O 设备。然后，CPU 继续执行其他任务，直到 I/O 设备完成数据的输入或输出后，向 CPU 发出中断请求信号。CPU 在接收到中断信号后，会暂停当前的任务，转而处理 I/O 设备的输入或输出请求。处理完成后，CPU 会继续执行原来的任务 1。

中断控制 I/O 方式相比程序直接控制 I/O 方式，可以提高 CPU 的利用率，减少 CPU 的等待时间。但是，中断控制 I/O 方式仍然需要 CPU 介入 I/O 操作，因此效率相对较低 1。

40 DMA 方式能够提高成批数据交换效率的主要原因何在？

DMA 传输数据不需要 CPU 直接控制，而是通过 DMA 控制器完成，这样 CPU 在数据传输过程中可以继续执行其他任务，从而实现了 CPU 和 I/O 设备之间的并行工作。

41 DMA 控制器在什么情况下发出中断请求信号？

DMA 控制器在数据块传输结束时发出中断请求信号

42 I/O 端口的编址方式有哪两种？各有何特点？

统一编址 和 独立编址

统一编址是将 I/O 端口与内存单元统一起来进行编号，不需要专门的 I/O 指令，对 I/O 端口操作的指令类型多，

但端口要占用部分存储器的地址空间，不容易区分是访问存储器还是外部设备 12。

独立编址是端口单独构成 I/O 地址空间，不占用存储器地址，地址空间独立，控制电路和地址译码电路简单，采用专用的 I/O 指令，使得端口操作的指令在形式上与存储器操作指令有明显区别，程序容易阅读，但指令类别少，一般只能进行传送操作

43 为什么中断控制器把中断类型型号放 I/O 总线的数据线上而不是放在地址线上

- ① **数据传输效率**：I/O 总线的数据线通常用于传输数据，其传输速度比地址线快。将中断类型号放在数据线上可以更快地传递给处理器，减少了中断响应的延迟时间。
- ② **地址线的用途**：地址线主要用于指定内存或 I/O 设备的地址。在中断处理过程中，地址线通常用于指向中断服务程序的入口地址（也称为中断向量表）。因此，将中断类型号放在地址线上会干扰地址的传输，不符合地址线的常规用途。
- ③ **简化设计**：将中断类型号放在数据线上可以简化中断控制器的设计。中断控制器只需要将数据线上的值读取出来，并将其传递给处理器，而不需要额外的逻辑来处理地址线。
- ④ **兼容性和标准化**：将中断类型号放在数据线上已经成为计算机体系结构的标准做法，这有助于保持硬件和软件之间的兼容性。不同的处理器和中断控制器都可以遵循这一标准来实现中断处理

1. 44 如何将多个 C 程序模块组合生成可执行文件？简述从源程序到可执行文件的转换过程

- ① 编写源代码：将各个模块的 C 源文件编写完毕。
- ② 预处理：使用 cpp 预处理器将#include 指令展开，#define 定义替换，去除注释，以及条件编译指令处理后生成.i 文件。
- ③ 编译：使用 ccl 编译器将预处理后的文件编译成汇编语言文件，即.s 文件。
- ④ 汇编：使用 as 汇编器将汇编语言文件转换成目标文件，即.o 文件。
- ⑤ 链接：使用 ld 链接器将多个目标文件及所需的库文件合并生成最终的可执行文件

2. 45 可重定位目标文件和可执行目标文件的主要差别是什么

- ① **可执行目标文件的.rel.text 和.rel.data 消失了**。链接器将.o 中.text 和.data 节整合到一起时，会对整合后的.text 和.data 进行重定位，.text 和.data 节重定位时需要依赖.rel.text 和.rel.data 中的信息，一旦重定位结束

后，这两个节的使命就完成了，所以可执行目标文件中不存在 `rel.text` 和 `rel.data` 节。

② **可执行目标文件多出了 `init` 节**。`init` 节的作用是初始化程序的堆栈。在编译时候，如果加上 `-v` 查看详细的编译过程，就会看到当程序加载到内存时，需要将硬盘上的可执行目标文件，搬到运行地址所指定的内存位置，这个过程需要段头部表中的信息辅助完成。

3. 46 链接器主要完成哪俩方面的工作？

符号解析：是将目标文件中定义和引用的符号，刚好和一个符号定义联系起来。

重定位是：编译器和汇编器生成从地址 0 开始的代码和数据节，链接器将每个符号定义与一个存储器位置联系起来，然后修改所有对这些符号的引用，使得它们指向这个存储器位置，从而重定位这些节。

4. 47 可重定位文件的 `.text` 节、`.rodata` 节、`.data` 节和 `.bss` 节中分别主要包含什么信息？

`.text` 节：

- ① 主要包含程序的机器代码指令。
- ② 这是一个只读段，因为它包含程序代码，一旦程序开始执行，这些指令就不会被修改。
- ③ 链接器在链接过程中会合并所有目标文件的 `.text` 节，并生成一个最终的 `.text` 节。

`.rodata` 节：

- ① 包含程序中的只读数据，例如常量字符串、全局常量等。
- ② 由于这些数据是只读的，所以它们被放置在单独的节中以优化内存使用。
- ③ 链接器在链接过程中会合并所有目标文件的 `.rodata` 节，并生成一个最终的 `.rodata` 节。

`.data` 节：

- ① 包含程序中初始化的全局变量和静态变量。
- ② 这些变量在程序开始执行之前由操作系统或运行时库初始化。
- ③ 链接器在链接过程中会合并所有目标文件的 `.data` 节，并生成一个最终的 `.data` 节。

`.bss` 节：

- ① 包含程序中未初始化的全局变量和静态变量。
- ② 这些变量在程序开始执行之前由操作系统或运行时库分配内存，但不会被初始化（即它们的初始值是未定

义的)。

③ 在链接过程中，链接器会合并所有目标文件的.bss 节，并生成一个最终的.bss 节。在加载可执行文件时，操作系统会为.bss 节分配相应的内存空间，并将其初始化为零。

5. 48 可执行目标中有哪两种可装入段？哪些节组合成只读代码段？哪些节组合成可读/写数据段？

代码段 和 数据段

在可执行目标文件中，以下节组合成只读代码段：

.text 节：包含程序的机器代码指令。

任何由链接器生成的重定位信息，这些信息用于在加载时修正代码段中的地址。

以下节组合成可读/写数据段：

.data 节：包含程序初始化的全局变量和静态变量。

.rodata 节：包含只读数据，如字符串常量。尽管它是只读的，但在某些上下文中，它可能被视为数据段的一部分，因为它与数据相关。

.bss 节：包含程序中未初始化的全局变量和静态变量。在加载时，操作系统会为其分配内存，并初始化为零。

虽然这个节在可重定位目标文件中不存在，但在加载可执行文件时，它会被视为数据段的一部分。

6. 48 加载可执行文件时，加载器根据其中哪个表的信息可对装入段进行映射？

可执行文件中的**段映射表 (Segment Mapping Table)** 或**节区头部 (Section Headers)** 的信息

7. 50 静态链接和动态链接的差别是什么？

- (1) 静态库浪费主存和磁盘空间：静态链接的代码始终被合并到可执行文件
- (2) 静态库更新困难，使用不便：静态链接需要定期维护更新静态库
- (3) 动态库的共享性和动态性：代码在内存只有一个副本，使用的程序在执行时才会加载到内存
- (4) 动态库的链接方式：程序加载过程中加载和连接、程序执行时加载和链接

8. 51 在可执行文件中将可装入段映射到虚拟地址空间，以形成每个进程独立的虚拟地址空间，这种做法有什么好处？

- (1) 地址隔离

- (2) 内存保护
- (3) 内存管理灵活性
- (4) 简化编程
- (5) 支持多道程序设计

9. 52 简述通过 Shell 命令行解释程序进行程序加载的过程

在 Shell 输入可执行文件名 a.out 简述程序加载的过程

- (1) Shell 输出提示符，接受用户输入的命令
- (2) 用户输入 ./a.out 回车之后，Shell 对命令进行解析，获取参数
- (3) 调用 fork 函数，创建子进程
- (4) 调用 execve，在当前进程中加载并运行 a.out

10. CPU 的基本组成和基本功能各是什么？

程序计数器 PC	指令寄存器 IR	指令译码器 ID
启停控制逻辑	时序信号产生部件	操作控制信号形成部件
总线控制逻辑	中断机构	

11. 如何控制一条指令执行结束后能够接着另一条指令执行

控制一条指令执行结束后能够接着另一条指令执行，通常涉及到程序的执行流程控制。这主要依赖于以下几个方面：

- ① **指令顺序：**在大多数程序中，指令是按照它们在代码中的顺序来执行的。因此，简单地将一条指令放在另一条指令之后，就可以保证前者执行结束后接着执行后者。
- ② **函数调用：**在函数调用中，当函数被调用时，控制权转移到函数内部，并按照函数内部的指令顺序执行。函数执行完毕后，控制权返回调用点，继续执行调用点之后的指令。
- ③ **条件跳转：**在某些情况下，可以使用条件跳转指令来改变程序的执行流程。这通常涉及到比较操作的结果，如果满足某个条件，则跳转到代码中的某个特定位置执行。例如，在 x86 架构的汇编语言中，可以使用 jmp、je（如果等于则跳转）、jne（如果不等于则跳转）等指令。

④ **循环结构**：循环结构（如 for、while、do-while 等）允许指令在满足特定条件时重复执行。循环体中的指令会按照顺序执行，直到循环条件不再满足为止。

⑤ **异常处理**：当程序遇到异常或错误时，通常会跳转到异常处理代码。这可以通过 try-catch 结构（在某些编程语言中）或设置异常处理函数来实现。

⑥ **中断和信号**：在操作系统层面，中断和信号可以用来控制程序的执行流程。当操作系统接收到中断或信号时，它可以暂停当前程序的执行，转而执行中断处理程序或信号处理函数。处理完毕后，控制权通常返回给被中断的程序，继续执行。

⑦ **线程调度**：在多线程环境中，线程的调度机制决定了哪个线程何时执行。尽管单个线程内部指令的执行顺序是确定的，但不同线程之间的执行顺序可能会因为线程调度而发生变化。

12. 通常一条指令的执行要经过哪些步骤？每条指令的执行步骤都一样吗？

取指令、译码、取数、执行、写回、更新 PC

每条指令执行步骤不完全相同

13. 流水线方式下，一条指令的执行时间缩短了还是加长了？程序的执行时间缩短了还是加长了

在流水线方式下，单条指令的执行时间并没有缩短，但程序的总体执行时间可能会缩短。这是因为流水线技术允许 CPU 同时处理多条指令的不同阶段，从而提高了指令的执行效率。

14. 具有什么特征的指令系统易于实现指令流水线？

(1) 固定长度的指令

(2) 简单的指令格式

(3) 指令之间没有数据依赖

(4) 较少的指令类型

(5) 指令执行时间相近

(6) 支持预测执行

支持乱序执行